📘 rdt3.0: Stop-and-Wait Protocol

Reliable Data Transfer (rdt3.0) ensures data is delivered **correctly over a noisy channel**, using stop-and-wait with retransmissions.

🔹 How It Works:

1. Sender transmits **one packet** and waits for its ACK before sending the next.
2. Receiver sends an **ACK** after correctly receiving the packet.
3. If the ACK is lost or a timeout occurs, the sender **retransmits** the packet.

🔹 Performance Limitation:

• Utilization is **very low** for high-speed or long-delay channels.
• Example:

- Transmission time = L/R
- Round-trip time = RTT
- Channel utilization $U = \frac{L/R}{RTT+L/R} \approx 0.00027$ → ~0.03%

  • **Conclusion:** rdt3.0 wastes bandwidth because the sender is idle while waiting for ACKs.

🧠 **Key Concept:** Stop-and-wait is simple but **limits the throughput**, especially when RTT is large compared to packet transmission time.

📘 Pipelined Protocols (rdt3.0 Improvement)

To **increase utilization**, pipelining allows the sender to transmit **multiple packets before receiving ACKs**.

🔁 How It Works:

1. Sender can have several **"in-flight" packets**.
2. Both sender and receiver maintain **buffers** to store unacknowledged or out-of-order packets.
3. Sequence numbers must be **larger**, enough to distinguish all packets in the pipeline.

🧠 Example:

- If 3 packets are in-flight:
  - First packet transmitted at t = 0 → ACK received at t = RTT + L/R
  - Second packet transmitted immediately after first → ACK arrives slightly later
  - Third packet sent immediately after second → ACK arrives
- **Result:** Utilization increases by factor of 3!

☑ **Final Note:**
Pipelining **dramatically improves throughput** over stop-and-wait, making better use of available bandwidth.

📘 **Go-Back-N (GBN)**
GBN is a pipelined protocol where the sender can transmit **multiple packets**, but on **loss, all subsequent packets are retransmitted**.

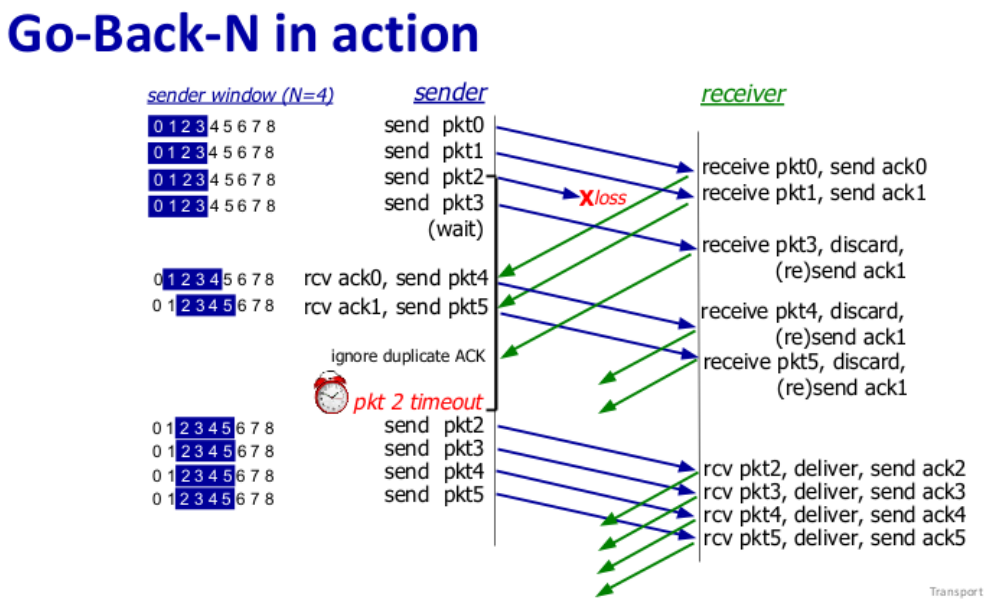
• Receiver only accepts **in-order packets**.
• Lost or corrupted packet → **discard all subsequent packets**.

🔄 **How It Works:**

1. Sender maintains a **window of N packets**.
2. Receiver sends **cumulative ACK** for the last in-order packet.
3. If timeout occurs → sender retransmits **all packets starting from the missing one**.

🧠 **Example:**

- Window = 4, sender transmits 0–3.
- Packet 2 lost → packets 3+ discarded by receiver.
- Timeout → retransmit 2,3.



☑ **Final Line:**
GBN ensures reliability but may **retransmit many packets unnecessarily** on a single loss.

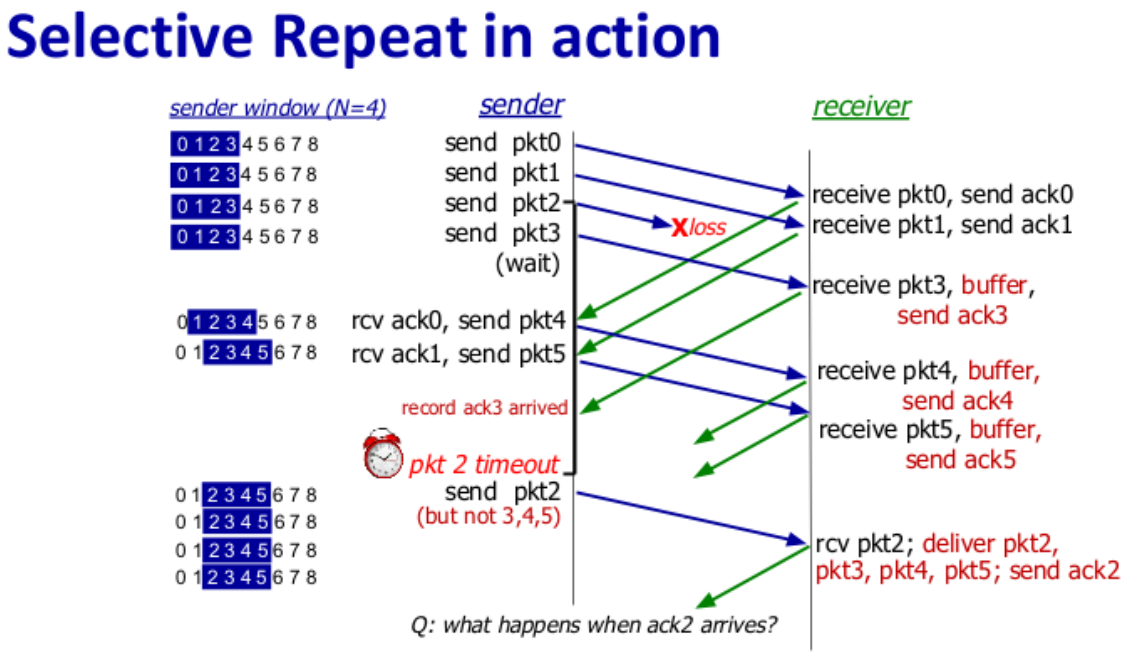
📘 **Selective Repeat (SR)**
SR improves GBN by **retransmitting only lost packets** and **buffering out-of-order packets**.

• Each packet is **individually acknowledged**.
• Receiver maintains a **window** to buffer out-of-order packets.
• Sender maintains a **timer per packet**, instead of one for the entire window.

🔁 **How It Works:**

1. Sender transmits packets within window size N.
2. Receiver sends **ACK for each correctly received packet**.
3. Lost packets are **retransmitted individually**.
4. Receiver delivers buffered packets **in order** once the missing packet arrives.

🧠 **Example:**



- Sender sends packets 0–5.
- Packet 2 lost → timeout → retransmit 2 only.
- Receiver buffers 3–5 → delivers all in order after 2 arrives.

☑ **Final Line:**
SR is **more efficient than GBN**, especially in networks with high loss or large windows.

📘 **Selective Repeat (SR) – Receiver Dilemma**

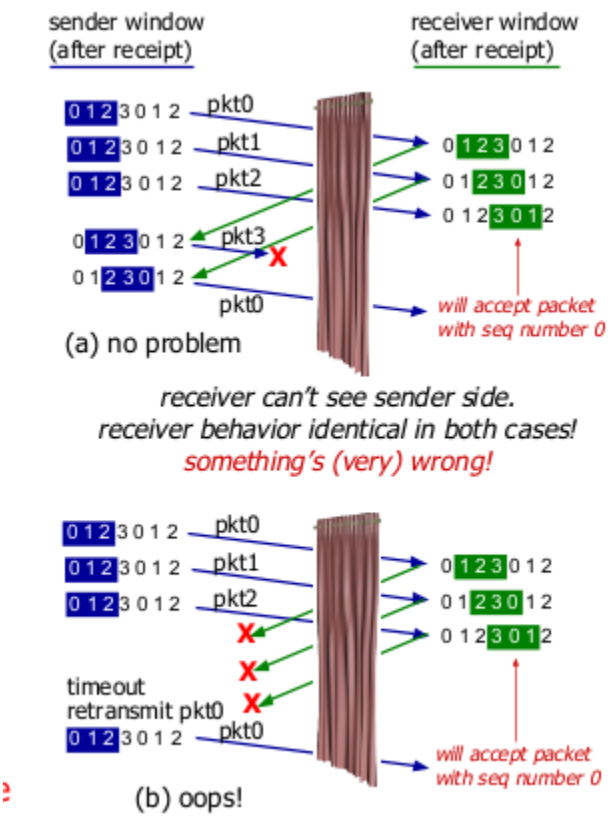
Selective Repeat allows the sender to **retransmit only lost packets**, and the receiver can **accept out-of-order packets** and buffer them until missing ones arrive.

🔁 **Scenario Example:**

- Sequence numbers: 0, 1, 2, 3
- Window size: 3

Sender and Receiver Windows After Packet Transmission:

Step	Sender Window	Receiver Window	Action
1	0 1 2	0 1 2	Send pkt0, pkt1, pkt2
2	1 2 3	0 1 2	pkt0 lost, pkt1 and pkt2 received
3	0 1 2	0 1 2	Timeout → retransmit pkt0
4	0 1 2	0 1 2	Receiver accepts pkt0 **again**



🧠 **Problem Observed:**

- Receiver can't distinguish between **old pkt0** and **retransmitted pkt0** if the **sequence numbers wrap around**.
- This can cause **duplicate data** to be delivered as new.

🔹 **Key Insight:**

- The **receiver cannot see the sender's window**; it relies solely on sequence numbers.
- If the **window size is too large relative to sequence number space**, duplicate packets may be incorrectly accepted.

📘 **Solution / Condition:**
To avoid the duplicate data problem:

Window size ≤ ½ of sequence number space

☑ **Reason:**

- Ensures that by the time sequence numbers wrap around, **all previous packets with that sequence number have been acknowledged** and delivered.
- Prevents the receiver from mistaking retransmitted packets for new data.

🧠 **Summary Line:**
Selective Repeat increases efficiency by sending only lost packets, but **sequence number space must be at least twice the window size** to avoid duplicate delivery problems.

📘 TCP Sequence Numbers and Acknowledgments

TCP treats data as a **continuous byte stream** rather than discrete messages or segments. Each byte in the stream is **numbered sequentially**, which allows TCP to keep track of **lost or out-of-order data**.

🔁 How It Works:

- Host A wants to send a file of **500,000 bytes** to Host B.
- Maximum Segment Size (MSS) = 1,000 bytes.
- TCP divides the stream into **segments** and assigns **sequence numbers** based on the **first byte** of the segment.

📘 TCP Segment Structure:

Field	Purpose
Source Port # / Destination Port #	Identify sending and receiving application processes
Sequence Number	Byte number of the first byte in this segment's data
Acknowledgment Number	Sequence number of the **next expected byte** from the other side
Data	Actual application data (variable length, ≤ MSS)
Receive Window (rwnd)	Flow control: # bytes receiver is willing to accept
Flags (SYN, FIN, RST, ACK, URG, CWR, ECE)	Connection management and control
Checksum	Error detection
Options	Variable-length features (optional)

🧠 **Key Concept – Sequence Numbers:**

- Every byte of data is assigned a **unique sequence number**.
- TCP segments carry the **sequence number of the first byte** in their payload.
- **In-flight data:** Sent but not yet acknowledged.
- **Usable sequence numbers:** Available for sending but not yet used.

📘 **Acknowledgments (ACKs):**

- ACK indicates the **next byte expected** from the sender.
- TCP uses **cumulative ACKs**, i.e., it acknowledges **all bytes up to, but not including, the next expected byte**.
- Example: If Host A has received bytes **0–535**, the ACK sent back will indicate **536** as the next expected byte.

🔹 **Handling Out-of-Order Segments:**

- Scenario: Host A receives segments 0–535 and 900–1000.
- Bytes 536–899 are missing → **gap in sequence**.
- Host A can either:
  - **Discard out-of-order segments** (simpler, less memory)
  - **Buffer them** (more efficient, ensures fast delivery when missing bytes arrive)
  - TCP spec allows both; implementation-dependent

🧠 **Summary Line:**
TCP sequence numbers ensure reliable, in-order delivery of byte streams. ACKs tell the sender which bytes the receiver has received, while out-of-order data can be buffered or discarded based on implementation.

📘 **TCP Sequence Numbers and ACKs – Simple Telnet Example**

TCP segments carry sequence numbers (Seq) and acknowledgments (ACK) to ensure **reliable, in-order delivery**.

🔄 **How It Works – Telnet Scenario:**

- Host A sends a character typed by the user (e.g., `'C'`) to Host B.

- Host B acknowledges receipt and echoes the character back.
- Example sequence:
  - Host A → Host B: Seq = 42, ACK = 79, data = `'C'`
  - Host B → Host A: Seq = 79, ACK = 43, data = `'C'`
  - Host A → Host B: Seq = 43, ACK = 80 (final ACK)

🧠 **Key Concept:**

- Each byte of data has a **sequence number**.
- ACKs confirm receipt **up to the next expected byte**.

📘 TCP Round-Trip Time (RTT) and Timeout

TCP must decide how long to **wait for an ACK** before assuming a segment is lost.

🔹 **Challenge:**

- RTT varies due to network congestion and load.
- **Too short timeout:** Premature retransmissions → wasted bandwidth
- **Too long timeout:** Slow reaction to lost segments → delays

🔄 **Estimating RTT:**

- **SampleRTT:** Time from sending a segment until receiving its ACK
- Ignore retransmissions to avoid bias
- Use **Exponential Weighted Moving Average (EWMA)** to smooth variations:

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

- Typical α = 0.125 (gives more weight to recent samples)

🔹 **Timeout Interval:**

- Add a safety margin for variation:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

- **DevRTT:** Smoothed deviation of SampleRTT from EstimatedRTT

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

- Typical β = 0.25

🧠 **Key Concept:**

- TCP dynamically adjusts timeout based on RTT measurements to balance **reliability and efficiency**.

📘 **TCP Retransmission Scenarios**

🔹 Scenario 1: Lost ACK

- Host A sends segment **Seq=92 (8 bytes)**.
- Host B receives it and sends back **ACK=100**, but this ACK is **lost** in the network.
- Host A doesn't know the ACK was lost, so after a while its **timer expires (timeout)**.
- Host A **retransmits Seq=92** unnecessarily.
- But by now Host B has already received **later data (Seq=100, 120 …)** and sends **ACK=120**, which **cumulatively covers the earlier segment**.
- Result: even though an ACK was lost, the **cumulative ACK mechanism prevents duplicate delivery** — TCP stays consistent.

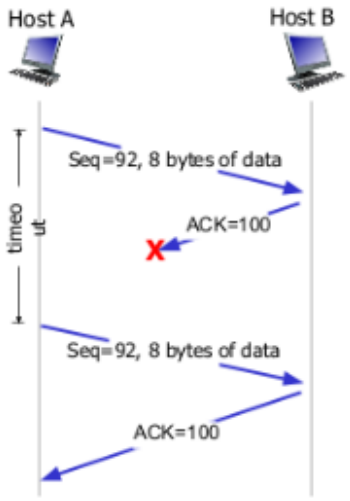
🧠 Key point: *Cumulative ACKs can "heal over" lost acknowledgments.*

🔹 Scenario 2: Cumulative ACK covers earlier lost ACK

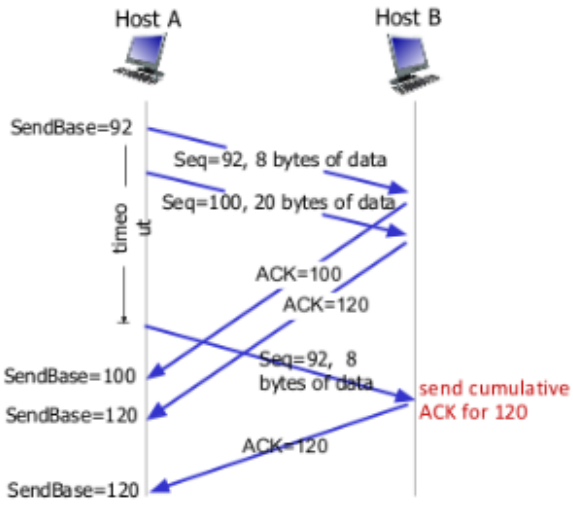
- Host A sends **Seq=92 (8 bytes)**, **Seq=100 (20 bytes)**, **Seq=120 (15 bytes)**.
- ACK for Seq=92 is **lost**.
- Later, Host B sends **ACK=120** (meaning: "I've got everything up to byte 119").
- This ACK **implicitly acknowledges** Seq=92 and Seq=100 as well.

🧠 Key point: *With cumulative ACKs, later ACKs cover earlier segments, so sender doesn't need per-segment ACKs.*

TCP: retransmission scenarios

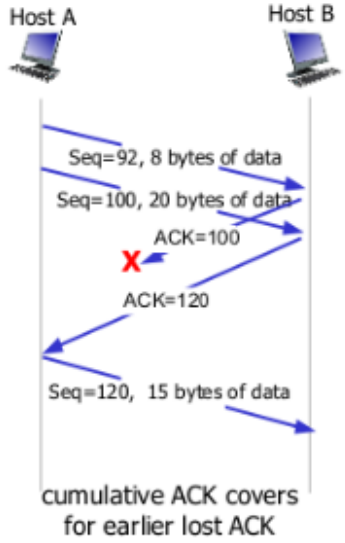


lost ACK scenario



premature timeout

TCP: retransmission scenarios



cumulative ACK covers for earlier lost ACK

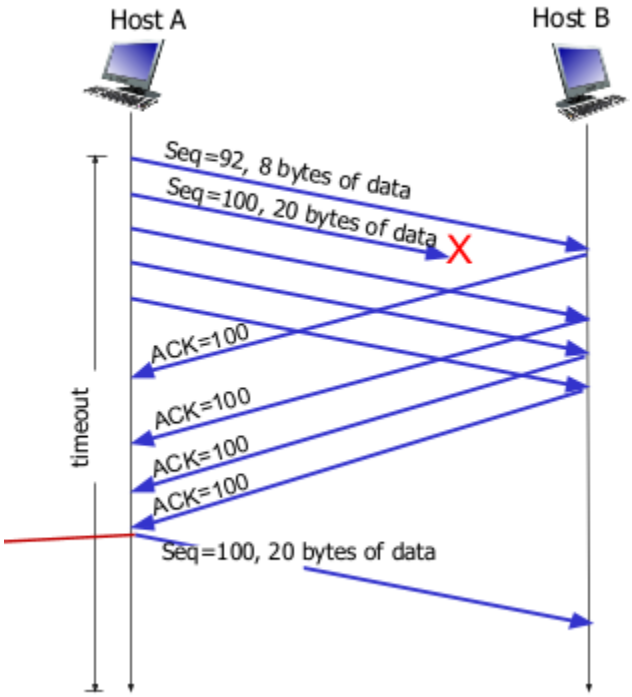
🌟 Why This Matters

- If TCP required an ACK for **every segment individually**, lost ACKs would constantly trigger wasteful retransmissions.
- Instead, TCP's **cumulative ACK rule** makes it robust:
  - ACK(n) = "I have all bytes up to n-1."
  - So one ACK can cover multiple earlier ones that got lost.

☑ Quick Check for You:
Imagine Host A sends **Seq=200 (MSS=1000 bytes)**, then **Seq=1200**, then **Seq=2200**.
If the **ACK for 200** is lost, but Host B later sends **ACK=3200**, what does this mean for the sender?

2. Fast Retransmit (Triple Duplicate ACKs)

- If a segment is lost, TCP may receive **duplicate ACKs** for the next in-order byte.
- **Rule:** On receiving **3 duplicate ACKs**, retransmit the missing segment **without waiting for timeout**.
- Example:
  - Host A sent segments Seq = 92, 100, 120
  - ACK = 100 received 3 times → segment 92 likely lost
  - Retransmit Seq = 92 immediately



🧠 **Key Concept:**

- TCP avoids waiting for timeout by using **cumulative ACKs and fast retransmit**, improving recovery speed.

🧠 **Summary Line:**
TCP uses sequence numbers and cumulative ACKs to track bytes, EWMA to estimate RTT for dynamic timeouts, and fast retransmit (triple duplicate ACKs) to quickly recover lost segments without waiting for timer expiration.

📘 **TCP Flow Control**
TCP prevents **receiver buffer overflow** using **rwnd (receive window)**.

• Sender limits **in-flight data ≤ rwnd**.
• Receiver advertises **available buffer space**.

🔄 **How It Works:**

1. Receiver sets `RcvBuffer` (default ~4 KB).
2. Advertises **free space in rwnd**.
3. Sender sends only as much data as allowed.

🧠 **Example:**

- rwnd = 2 KB → sender can send 2 KB unACKed.
- rwnd = 0 → sender stops until receiver frees buffer.

☑ **Final Line:**
TCP flow control guarantees **safe delivery** without overflowing receiver buffers.

📘 **TCP Flow Control**

🔹 **The Problem:**

What if the **sender** pushes data into the network **faster** than the **receiver's application** can read from its socket buffer?

👉 The receiver's buffer could **overflow**, causing data loss.

🔹 **The Solution: Flow Control**

TCP uses a **receiver-advertised window (rwnd)** to make sure the sender never overwhelms the receiver.

- Each TCP segment header has a **rwnd field**.
- `rwnd` tells the sender how much buffer space is still available at the receiver.
- The sender then **limits the amount of unACKed ("in-flight") data** so that:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$$

This guarantees that the receiver's buffer will not overflow.

🔹 **How the Receiver Updates rwnd:**

- **RcvBuffer:** total buffer space (e.g., 4096 bytes default).
- **LastByteRcvd:** highest byte received from the network.
- **LastByteRead:** highest byte read by the application.

So:

$$\text{rwnd} = \text{RcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead})$$

This means the receiver keeps advertising how much "room" is left in its buffer.

🔹 **Special Case: rwnd = 0**

- If the buffer is full, the receiver advertises `rwnd = 0`.
- Sender must stop sending — but it doesn't stay silent forever.
- Sender still transmits **1-byte "probe segments"** to check if the window has opened again.
- The receiver acknowledges these probes, possibly with a new, larger rwnd.

🧠 **Key Difference from Congestion Control:**

- **Flow control** = protects the receiver (end host) from being overwhelmed.
- **Congestion control** = protects the network from being overwhelmed.

☑ **Quick Recap (Mnemonic):**

- **rwnd** = **receiver's window** (flow control)
- **cwnd** = **congestion window** (congestion control)
- Sender is limited by the smaller of the two:

$$\text{AllowedWindow} = \min(\text{rwnd}, \text{cwnd})$$

📘 **TCP Connection Management**

TCP is **connection-oriented**, using a **3-way handshake** to establish connections.

• Establish connection → agree on **initial seq numbers, buffer sizes**.
• Close connection → send **FIN**, acknowledge FINs.

📘 **TCP 3-Way Handshake**

🔹 **Goal:**

Before sending data, both hosts must:

1. **Agree on initial sequence numbers (ISN)**
2. **Synchronize state** (so each side knows what to expect)
3. **Confirm both ends are ready** for communication

🔹 **The Three Steps:**

1. **SYN (synchronize)**
   - Client → Server
   - Client picks a random **initial sequence number (ISN)**, say `x`.
   - Sends a segment:
   - `Seq = x, SYN = 1`
   - This says: "I want to connect, and my sequence numbers start at `x`."

---

1. **SYN-ACK (synchronize + acknowledge)**
   - Server → Client
   - Server picks its own ISN, say `y`.
   - Acknowledges client's SYN by sending:
   - `Seq = y, Ack = x+1, SYN = 1, ACK = 1`
   - This says: "Got your SYN, my ISN is `y`, and I expect your next byte to be `x+1`."

---

1. **ACK**
   - Client → Server
   - Acknowledges server's SYN:
   - `Seq = x+1, Ack = y+1, ACK = 1`
   - Now both sides are synchronized!
   - After this, data transfer can begin.

🔹 **Visual Timeline:**

```
Client (Active Open)                Server (Passive Open)

   SYN, Seq = x ------------------->

                              <--- SYN-ACK, Seq = y, Ack = x+1

   ACK, Seq = x+1, Ack = y+1 ------->
```

🔹 **Why 3 steps?**

- Why not just 2? Because both sides need to **synchronize sequence numbers**.
- The third ACK ensures the client really got the server's SYN.

🔹 **Fun fact:**
There's also a **TCP 4-way handshake** for **connection termination**, but that's a slightly different process.

☑ **Quick Recap Mnemonic:**

- **SYN → SYN-ACK → ACK**
- "You there?" → "Yes, I'm here, are you ready?" → "I'm ready, let's talk."

📘 TCP Closing Connection (4-Way Handshake)

🔹 Why 4 steps?

Because TCP is **full-duplex**: data can flow both ways. Each side must close **its own sending stream** separately.

🔹 The Four Steps:

1. **FIN (Finish)**
   - Suppose the **client** is done sending data.
   - It sends a segment:
   - `FIN = 1, Seq = u`
   - This says: "I've finished sending."

---

1. **ACK (Acknowledgment of FIN)**
   - Server receives FIN, and replies:
   - `ACK = u+1`
   - Now the server knows the client is done sending, but the server may still send data back.

---

1. **FIN (Server finishes)**
   - When the server is also ready to close, it sends:
   - `FIN = 1, Seq = v`
   - This says: "I'm also done sending."

---

1. **ACK (Final Acknowledgment)**
   - Client responds:
   - `ACK = v+1`
   - Now both sides know the other is finished. Connection is closed.

🔹 Visual Timeline:

```
Client (active close)                    Server (passive close)

  FIN, Seq=u ----------------------------->

                              <----------- ACK=u+1
```

```
                                <---------- FIN, Seq=v

ACK=v+1 ------------------------------->
```

🔹 **Extra detail — TIME_WAIT state**

- After the client sends the last ACK, it goes into **TIME_WAIT** (usually 2× Maximum Segment Lifetime).
- Purpose: to ensure any delayed/duplicate packets in the network don't interfere with a new connection.

☑ **Quick Recap Mnemonic:**

- **FIN → ACK → FIN → ACK**
- Think of it like politely hanging up a phone:
  - "I'm done."
  - "Got it."
  - "I'm done too."
  - "Okay, call's over."

☑ **Final Line:**
TCP connection management ensures **reliable setup and teardown**.

🌐 First: Why congestion control at all?

- In flow control, we protect the **receiver** from being overwhelmed.
- In congestion control, we protect the **network** from being overwhelmed.
- Too much sending → routers' queues overflow → packet loss, delays.

TCP does this by dynamically adjusting its sending rate with a strategy called **AIMD**.

📘 TCP Congestion Control: AIMD (Additive Increase, Multiplicative Decrease)

1. **Congestion Window (cwnd):**
   - The sender maintains a variable `cwnd` = how many bytes it is allowed to have "in flight" (unacknowledged).

- Works along with flow-control window (`rwnd`).
- Sender can send up to `min(cwnd, rwnd)`.

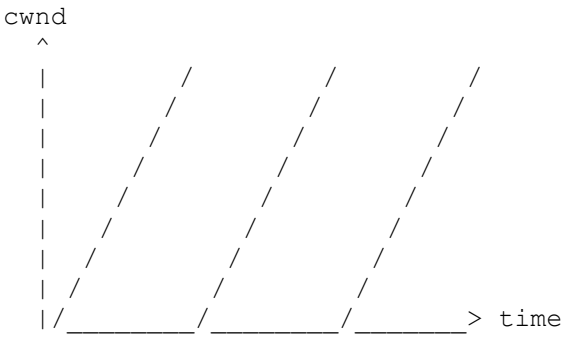
---

1. **Additive Increase (probing phase):**
   - As long as ACKs arrive normally, every **RTT**, the sender increases `cwnd` by **1 MSS**.
   - This is like gently pushing the gas pedal: "Let me see if I can send a bit faster."
   - Shape: slow upward slope of the sawtooth.

---

1. **Multiplicative Decrease (reaction to loss):**
   - If loss is detected:
     - **Triple duplicate ACKs (fast retransmit, TCP Reno):**
       - `cwnd = cwnd/2` (cut in half).
     - **Timeout (TCP Tahoe):**
       - `cwnd = 1 MSS` (reset almost to zero).
   - This is like slamming the brakes when you notice congestion.
   - Shape: steep downward drop of the sawtooth.

📊 Visualization (the "sawtooth")



- Slopes upward slowly (additive increase).
- Drops suddenly (multiplicative decrease).
- Repeats → a sawtooth pattern.

☑ Why AIMD?

- **Fairness:** if multiple TCP flows share a bottleneck, AIMD tends to equalize their rates.

- **Stability:** avoids congestion collapse (network grinding to a halt due to too many retransmissions).
- **Efficiency:** probes for available bandwidth without permanently overloading the network.

📘 TCP Congestion Control: AIMD

TCP prevents network congestion using **Additive Increase, Multiplicative Decrease (AIMD)**.

• Increase cwnd gradually (additive) until loss occurs.
• On loss → decrease cwnd (multiplicative).
• Creates a **sawtooth pattern**: probe bandwidth → reduce rate → repeat.

🔄 How It Works:

1. Sender maintains **cwnd** (congestion window).
2. Sending rate ≈ cwnd / RTT.
3. Additive increase: +1 MSS per RTT.
4. Multiplicative decrease: cut cwnd by half on triple duplicate ACK, or cwnd = 1 MSS on timeout.

🧠 Example:

- cwnd = 10 MSS → triple duplicate ACK → cwnd = 5 MSS.
- Timeout → cwnd = 1 MSS.

☑ Final Line:

AIMD balances **efficient bandwidth usage** and **network stability**.

🌱 What is Slow Start?

- **Problem:** At the very beginning of a TCP connection, the sender doesn't know the available bandwidth.
- **Naive sending:** If it starts too aggressively, it may flood the network.
- **Too timid:** If it sends too little, it wastes bandwidth.

Solution = Slow Start

- Begin cautiously, then quickly ramp up sending rate.

⚙️ How Slow Start Works

1. **Initialization:**
   - `cwnd = 1 MSS` (just one segment in flight).
2. **Exponential Growth:**
   - For every ACK received, `cwnd` increases by **1 MSS**.
   - Since all packets in `cwnd` get ACKed, this effectively **doubles cwnd every RTT**.
   - Pattern: 1, 2, 4, 8, 16… segments.
3. **Stopping condition:**
   - Stop exponential growth once `cwnd >= ssthresh` (slow start threshold).
   - After that, switch to **Congestion Avoidance (CA)**, where `cwnd` grows linearly (Additive Increase).

🚦 What happens when loss occurs?

- **Loss detected by timeout (severe congestion):**
  - `cwnd = 1 MSS` (back to the beginning).
  - `ssthresh = cwnd/2` (where cwnd was just before the loss).
  - Start over with exponential growth.
- **Loss detected by 3 duplicate ACKs (milder congestion):**
  - Enter **Fast Recovery** (skip going all the way back to 1).

📊 Example Timeline (ideal case, no loss at first)

- RTT 1: `cwnd = 1 MSS` → send 1 segment.
- RTT 2: ACK returns → `cwnd = 2 MSS` → send 2 segments.
- RTT 3: ACKs return → `cwnd = 4 MSS` → send 4 segments.
- RTT 4: ACKs return → `cwnd = 8 MSS` → send 8 segments.
- … doubles until `cwnd >= ssthresh`.

💡 **Analogy:** Imagine merging onto a highway. You start slow, but each time you check (ACK), you press the gas harder (doubling speed). Once you reach a "safe cruising speed" (ssthresh), you stop doubling and only accelerate gradually (Congestion Avoidance).

☑ Quick check for you:
Suppose `ssthresh = 32 MSS`. If the connection starts with `cwnd = 1 MSS`, what will `cwnd` be after **5 RTTs** (assuming no loss)?

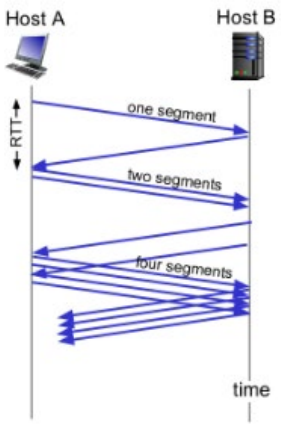
📘 TCP Slow Start

TCP Slow Start is the mechanism **used when a connection begins** to avoid overwhelming the network.

- **Goal:** Ramp up the sending rate **quickly but safely**.
- **Initial congestion window (cwnd) = 1 MSS** (Maximum Segment Size).
- **Exponential growth:** cwnd doubles **every RTT** until a loss is detected.

🔄 How It Works:

1. Start with `cwnd = 1 MSS`.
2. Send 1 segment, wait for ACK.
3. On receiving ACK → increase `cwnd` by 1 MSS.
4. Next RTT → send 2 segments, then 4, 8, …
5. Growth continues until **first packet loss** (timeout or triple duplicate ACK).



🧠 Example (Analogy):

- Imagine **filling a cup of water from a tap**:
  - Start slow → 1 drop.
  - After observing that it flows well, double the flow each round.
  - Stop doubling when the cup starts overflowing (network loss).

☑ **Final Line:**
TCP Slow Start allows the sender to **quickly ramp up throughput** without immediately overloading the network.

📘 **TCP Loss Handling in Slow Start**

1. **Loss detected by timeout:**
   - cwnd reset to **1 MSS**.
   - Threshold `ssthresh = cwnd/2` (value of cwnd just before loss).
   - Restart slow start to ssthresh, then switch to linear growth.
2. **Loss detected by 3 duplicate ACKs (Fast Recovery):**
   - cwnd reduced **less drastically** than timeout.
   - TCP Reno: cwnd = cwnd / 2 → then grows **linearly**.
   - TCP Tahoe: cwnd reset to 1 MSS → grows **exponentially** (slow start).

🧠 **Example:**

- cwnd = 16 MSS, loss occurs.
- Timeout → cwnd = 1 MSS, ssthresh = 8 MSS.
- Triple duplicate ACKs → TCP Reno: cwnd = 8 MSS, grows linearly.
- TCP Tahoe: cwnd = 1 MSS, slow start again.

☑ **Final Line:**
TCP uses **different reactions to loss** depending on whether it's a **timeout** or **triple duplicate ACKs**.

📘 **TCP Congestion Avoidance (CA)**
After slow start reaches **ssthresh**, TCP enters **congestion avoidance**, growing **linearly instead of exponentially**.

• **Goal:** Probe the network more cautiously after initial ramp-up.
• **cwnd increases by 1 MSS per RTT**, instead of doubling.
• Ensures **steady growth** to avoid triggering congestion.

🔄 How It Works:

1. cwnd ≥ ssthresh → switch from **exponential (slow start) to linear (CA)**.
2. Increment cwnd slowly:

$$\text{cwnd} = \text{cwnd} + \frac{\text{MSS}}{\text{cwnd}} \text{ per ACK received}$$

3. Continue linear growth until **loss event** occurs (timeout or triple duplicate ACKs).

🧠 Example (Analogy):

- Imagine **driving uphill**:
  - Slow start = gas pedal fully pressed → speed doubles quickly.
  - Congestion avoidance = ease off the pedal → speed increases gradually to avoid skidding.

☑ Final Line:

TCP Congestion Avoidance ensures the sender **probes for extra bandwidth carefully** without overwhelming the network.

🚦 Congestion Avoidance (CA) in TCP

Goal: Avoid overloading the network after the initial "ramp-up" phase.

⚙ How CA Works

1. **Linear growth instead of exponential:**
   - Instead of doubling `cwnd` every RTT (like in slow start), **increase cwnd by 1 MSS per RTT**.
   - Formula: `cwnd = cwnd + (MSS * (MSS / cwnd))` effectively gives linear growth.
2. **When to switch from Slow Start to CA:**
   - When `cwnd >= ssthresh` (slow start threshold).

- Threshold is usually set to **half of cwnd** just before a congestion event (loss).

3. **What triggers congestion detection:**
   - **Timeout:** severe congestion → reset `cwnd` (TCP Tahoe: cwnd → 1 MSS)
   - **Triple duplicate ACKs:** mild congestion → fast recovery (TCP Reno: cwnd → cwnd / 2, then linear growth)

📊 TCP Versions Behavior

Version	Timeout Event	Triple Duplicate ACKs
Tahoe	cwnd → 1 MSS	cwnd → 1 MSS
Reno	cwnd → 1 MSS	cwnd → cwnd / 2 + 3 MSS (fast recovery)

🧠 Key Concept

- **Slow Start:** exponential growth to quickly utilize available bandwidth.
- **Congestion Avoidance:** linear growth to probe for more bandwidth safely.
- **Fast Recovery:** handles mild congestion without dropping cwnd to 1.

💡 Analogy:

- Slow Start = sprint to get on the highway.
- Congestion Avoidance = cruise carefully, adding speed slowly to avoid traffic jams.
- Fast Recovery = small detour after noticing a traffic jam but don't stop completely.

☑ Check Understanding:

- Suppose `ssthresh = 16 MSS` and after Slow Start `cwnd = 16 MSS`.
- How will `cwnd` grow in **the next 3 RTTs** during Congestion Avoidance if no loss occurs?

📘 TCP Fast Recovery

• Triggered by **3 duplicate ACKs** (segment lost but network still delivering some segments).
• Reduces cwnd **less drastically than timeout**, allowing **faster recovery**.

• TCP Reno: cwnd = cwnd / 2 → linear growth.
• Allows **network to continue delivering segments**, instead of stopping completely.

🔁 **How It Works:**

1. Detect **3 duplicate ACKs** → enter fast recovery.
2. Retransmit **lost segment immediately**.
3. cwnd cut in half → **linear increase** resumes.
4. Avoids restarting slow start unnecessarily.

🧠 **Example:**

- cwnd = 16 MSS.
- Segment lost → 3 duplicate ACKs received.
- TCP Reno: cwnd = 8 MSS, retransmit missing segment, then grow linearly.

☑ **Final Line:**
Fast Recovery allows TCP to **recover from minor losses quickly** without returning to slow start.

📘 **TCP: From Slow Start to Congestion Avoidance**

• **Question:** When should exponential growth switch to linear?
• **Answer:** When cwnd reaches **ssthresh** (usually half of cwnd before last loss).

• **Implementation:**

- `ssthresh` = 1/2 of cwnd before loss.
- On loss:
  - Timeout → cwnd = 1 MSS → slow start.
  - Triple duplicate ACKs → Reno: cwnd = cwnd/2 → linear growth.

🧠 **Example (Analogy):**

- Climbing a hill: start fast → slow down near the top to **avoid slipping** (linear growth = congestion avoidance).

☑ **Final Line:**
TCP combines **slow start, congestion avoidance, and fast recovery** to maximize throughput while avoiding congestion collapse.

📘 **TCP Congestion Control: Overview**
TCP congestion control ensures that **senders do not overwhelm the network**, adjusting the sending rate based on observed network conditions.

• Congestion occurs when packets are **lost or delayed** due to network overload.
• TCP adapts by **increasing or decreasing the congestion window (cwnd)** using different strategies.
• **Key mechanisms:** Slow Start, Congestion Avoidance, Fast Recovery, and AIMD.

🔄 **TCP Congestion Window Behavior:**

1️⃣ **Slow Start (cwnd < ssthresh)**
• Window grows **exponentially**: cwnd doubles every RTT.
• Purpose: Ramp up sending rate quickly at connection start.
• On **ACK received** → cwnd = cwnd + 1 MSS.

🧠 Example:

- Start with cwnd = 1 MSS → ACK received → cwnd = 2 MSS → next RTT → cwnd = 4 MSS → …

2️⃣ **Congestion Avoidance (cwnd ≥ ssthresh)**
• Window grows **linearly**: cwnd increases by MSS/cwnd for every ACK.
• Purpose: Probe network slowly to avoid congestion.

🧠 Example:

- cwnd = 10 MSS, ssthresh = 8 MSS → next ACK → cwnd = 10 + MSS/10 ≈ 10.1 MSS → continues linear growth.

3️⃣ Triple Duplicate ACK (Fast Recovery)

• Indicates a **segment likely lost**, but network is still delivering some packets.
• Actions:

- ssthresh = cwnd / 2
- cwnd = ssthresh + 3 MSS
- Retransmit missing segment immediately
  • Window grows **linearly** until new ACK confirms all segments.

🧠 Example:

- cwnd = 16 MSS → triple duplicate ACK → ssthresh = 8 MSS, cwnd = 11 MSS → retransmit lost segment.

4️⃣ Timeout Event

• Indicates **severe congestion** or segment loss.
• Actions:

- ssthresh = cwnd / 2
- cwnd = 1 MSS (restart slow start)
- dupACKcount = 0
- Retransmit missing segment

🧠 Example:

- cwnd = 20 MSS → timeout → ssthresh = 10 MSS, cwnd = 1 MSS → slow start resumes.

📊 Summary Table of TCP Congestion Control Actions

Event	ssthresh	cwnd	Action
Slow Start (ACK received)	Unchanged	cwnd + 1 MSS	Exponential growth
Congestion Avoidance (ACK)	Unchanged	cwnd + MSS/cwnd	Linear growth
Triple Duplicate ACK	cwnd / 2	ssthresh + 3 MSS	Retransmit missing segment, linear growth
Timeout	cwnd / 2	1 MSS	Retransmit missing segment, restart slow start

---

🧠 **Key Points:**
• **Slow Start** → cwnd < ssthresh → exponential growth
• **Congestion Avoidance** → cwnd ≥ ssthresh → linear growth
• **Triple Duplicate ACK** → moderate congestion → fast recovery
• **Timeout** → severe congestion → slow start

☑ **Final Note:**
TCP congestion control dynamically adjusts the sending rate using **Slow Start, Congestion Avoidance, and Fast Recovery** to **maximize throughput while preventing congestion collapse**.

TCP Tahoe vs TCP Reno (Easy Version)

1 TCP Tahoe (Old School, Simple)

When it reacts:

- Anytime TCP **detects a loss**, whether by **timeout** or **3 duplicate ACKs**.

How it reacts:

1. Cut the congestion window (cwnd) **drastically** → back to **1 MSS**.
2. Set `ssthresh = cwnd / 2` (just before loss).
3. Start **Slow Start** again → cwnd grows **exponentially** until it reaches ssthresh.
4. Then switch to **linear growth** (Congestion Avoidance).

Analogy:

- Imagine running on a highway: if you see traffic, **you slam the brakes completely** and start accelerating slowly again.

Key Point:

- Tahoe is safe but slow to recover because it **always restarts Slow Start** after a loss.

---

2 TCP Reno (Smarter, Modern)

When it reacts:

- Timeout → behaves like Tahoe.
- **3 duplicate ACKs** → triggers **Fast Recovery**.

How it reacts (Fast Recovery):

1. Set `ssthresh = cwnd / 2`.
2. Set `cwnd = ssthresh + 3 MSS` (**+3 MSS because 3 dup ACKs indicate some segments are still arriving**).
3. Retransmit the lost segment **immediately**.
4. For every new ACK, **increase cwnd slowly** (linear growth) until all outstanding segments are acknowledged.
5. Then continue with **Congestion Avoidance**.

Analogy:

- You're running on a highway: you see a small traffic jam, **slow down but don't stop completely**, wait for the jam to clear, then continue gradually.

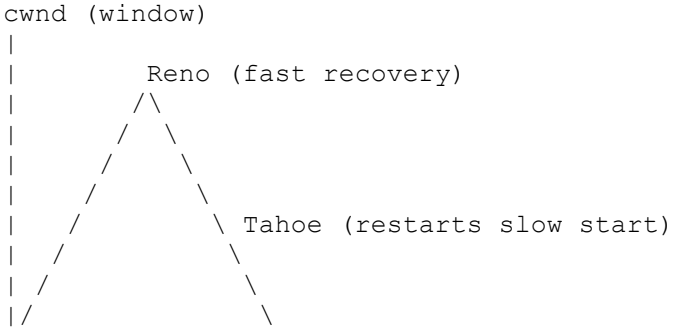
Key Point:

- Reno recovers **faster from minor packet loss** because it doesn't restart Slow Start for 3 duplicate ACKs.

Student-Friendly Comparison Table

Feature	Tahoe	Reno
Timeout	cwnd → 1 MSS, restart SS	cwnd → 1 MSS, restart SS
Triple duplicate ACKs	cwnd → 1 MSS, restart SS	cwnd → ssthresh + 3 MSS, Fast Recovery
Recovery Speed	Slow	Fast (for minor losses)
Growth after loss	Exponential → Linear	Linear (after Fast Recovery)

Simple Graph Idea



```
+------------------> time
```

☑ **Takeaway:**

- **Use Tahoe**: safe, simple, older networks.
- **Use Reno**: modern, efficient, handles minor losses faster.
- **Timeout = serious congestion**, **3 dup ACKs = minor loss**.